

CODE

```
import cv2

from ultralytics import YOLO
from pydantic import BaseModel
from typing import List

class DetectionResult(BaseModel):
    object_type: str
    confidence: float
    bbox: List[float] # [x_min, y_min, x_max, y_max]

class AegisXInference:
    def __init__(self,
        model_path="models/aegis_yolo_v8.pt"):
        # Load the custom fine-tuned YOLOv8 model
        self.model = YOLO(model_path)
        self.threat_classes = ['pistol', 'rifle', 'knife']

    def analyze_frame(self, frame) -> List[DetectionResult]:
        results = self.model(frame, conf=0.65) # 0.65 Threshold
        detections = []

        for result in results:
            for box in result.boxes:
                class_id = int(box.cls[0])
                label = self.model.names[class_id]
```

```
if label in self.threat_classes:
    detections.append(DetectionResult(
        object_type=label, confidence=float(box.conf[0]),
        bbox=box.xyxy[0].tolist()
    ))
return detections
```

```
from typing import TypedDict, Annotated, List
from langgraph.graph import StateGraph, END
from groq import Groq
```

```
# Define the shared state schema
```

```
class AgentState(TypedDict):
```

```
    threat_data: dict
```

```
    research_findings: str
```

```
    risk_score: int
```

```
    next_step: str
```

```
def supervisor_node(state: AgentState):
```

```
    client = Groq(api_key="your_groq_key")
```

```
    # LLaMA 3.3 70B analyzes the context
```

```
    response = client.chat.completions.create(
```

```
        model="llama-3.3-70b-versatile", messages=[{"role":
```

```
            "system", "content": "Analyze threat and delegate task."}]
```

```
    )
```

```
    # Decision logic to route to Researcher or Responder
```

```
    return {"next_step": "context_analyst"}
```

```
# Building the Graph
```

```
workflow = StateGraph(AgentState)
```

```
workflow.add_node("supervisor", supervisor_node)
workflow.set_entry_point("supervisor")
workflow.add_edge("supervisor", END)
```

```
app = workflow.compile()
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
import asyncio
import json
```

```
app = FastAPI(title="Aegis-X Command Center")
```

```
async def threat_event_generator(): """Simulates the SSE
stream of AI reasoning and detections.""" while True:
# In a real scenario, this pulls from a message queue
(Redis/RabbitMQ)
data = {
"agent": "Supervisor", "thought": "Weapon detected at
Sector 4. Initiating deep scan...", "severity": "CRITICAL"
}
yield f"data: {json.dumps(data)}\n\n" await
asyncio.sleep(0.1) # Maintain sub-100ms updates
```

```
@app.get("/v1/stream/intelligence")
async def stream_intelligence(request: Request):
return StreamingResponse(threat_event_generator(),
media_type="text/event-stream")
from datetime import datetime, timedelta
from jose import jwt
```

```
SECRET_KEY = "AEGIS_X_DEFENSE_PROTOCOL"
ALGORITHM = "HS512"
```

```
def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=60)
    to_encode.update({"exp": expire, "role":
"RESPONDER"})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY,
algorithm=ALGORITHM)
    return encoded_jwt
```

```
# Verify Token Middleware logic
def verify_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithm=[ALGORITHM])
        return payload if payload["role"] == "ADMIN" else None
    except jwt.JWTError:
        return None

def stream_pipeline(question: str) -> Generator[Tuple[str,
dict], None, None]:
    graph = _get_graph()
    for chunk in graph.stream(_initial_state(question),
stream_mode="updates"):
        for node_name, delta in chunk.items():
            yield node_name, delta

#config.py
import os
import re
```

```

import time

from dotenv import load_dotenv

from google import genai as google_genai
from google.genai import types as google_types
from langchain_core.messages import AIMessage,
BaseMessage, HumanMessage, SystemMessage


load_dotenv() _GOOGLE_API_KEY =
os.environ.get("GOOGLE_API_KEY", "")
_TAVILY_API_KEY =
os.environ.get("TAVILY_API_KEY", "")


if not _GOOGLE_API_KEY:
raise KeyError( "GOOGLE_API_KEY is missing. "
"Get a free key at aistudio.google.com and add it to .env:
GOOGLE_API_KEY=..."
)

if not _TAVILY_API_KEY:
raise KeyError("TAVILY_API_KEY is missing. Add it to
.env.") _client = google_genai.Client(
api_key=_GOOGLE_API_KEY,
http_options={"api_version": "v1alpha"},
) _MODEL_MAP : dict[str, str] = {
"junior_1": "gemini-3.1-flash-lite-preview", "junior_2":
"gemini-3.1-flash-lite-preview", "junior_3":
"gemini-3.1-flash-lite-preview", "supervisor":
"gemini-2.5-flash-lite", "hypothesis_refiner":
"gemini-3-flash-preview", "protocol_designer":
"gemini-3-flash-preview", "peer_reviewer":
"gemini-3.1-flash-lite-preview", "safety_officer":

```

```

"gemini-3.1-flash-lite-preview", "principal_investigator":
"gemini-3-flash-preview", "evaluator":
"gemini-3-flash-preview",
}_THINKING_BUDGET_MAP: dict[str, int] = {
"junior_1": 0, "junior_2": 0, "junior_3": 0, "supervisor":
1024, "hypothesis_refiner": 2048, "protocol_designer":
1024,
"peer_reviewer": 1024, "safety_officer": 1024,
"principal_investigator": 2048, "evaluator": 1024,
}_JSON_MODE_MAP: dict[str, bool] = {
"junior_1": False, "junior_2": False, "junior_3": False,
"supervisor": True, "hypothesis_refiner": True,
"protocol_designer": True, "peer_reviewer": True,
"safety_officer": True, "principal_investigator": True,
"evaluator": True, }_TEMPERATURE_MAP: dict[str,
float] = {
"junior_1": 0.1, "junior_2": 0.1,
"junior_3": 0.15, "supervisor": 0.1, "hypothesis_refiner":
0.15, "protocol_designer": 0.05, "peer_reviewer": 0.05,
"safety_officer": 0.05, "principal_investigator": 0.05,
"evaluator": 0.1, }_MAX_TOKENS_MAP: dict[str, int] =
{
"junior_1": 2048, "junior_2": 2048, "junior_3": 2048,
"supervisor": 4096, "hypothesis_refiner": 4096,
"protocol_designer": 3000, "peer_reviewer": 2048,
"safety_officer": 2048, "principal_investigator": 4096,
"evaluator": 3000, }

_MODEL_COSTS: dict[str, tuple[float, float]] = {
"gemini-2.5-flash-lite": (6.375 / 1_000_000, 25.5 /

```

```

1_000_000), "gemini-2.5-flash": (12.75 / 1_000_000, 51.0
/ 1_000_000), "gemini-3.1-flash-lite-preview": (6.375 /
1_000_000, 25.5 / 1_000_000), "gemini-3-flash-preview":
(12.75 / 1_000_000, 51.0 / 1_000_000),
}_DEFAULT_COST: tuple[float, float] = (12.75 /
1_000_000, 51.0 / 1_000_000)

```

```

def calculate_cost(prompt_tokens: int, completion_tokens:
int, model: str = "") -> float:
    input_rate, output_rate = _MODEL_COSTS.get(model,
_DEFAULT_COST)
    return round(prompt_tokens * input_rate +
completion_tokens * output_rate, 6)

```

```

def _retry_sleep(attempt: int, error: Exception) -> None:
    wait = min(2 ** attempt, 60)
    match = re.search(r"try again in ([\d.]+)(ms|s)", str(error))
    if match:
        val, unit = float(match.group(1)), match.group(2)
        wait = max(wait, (val / 1000) if unit == "ms" else val) +
0.5
    if any(k in str(error).lower() for k in ["503", "unavailable",
"high demand"]):
        wait = max(wait, 10 * (attempt + 1))
    time.sleep(wait)

class _Response: __slots__ = ("content",
"prompt_tokens", "completion_tokens")

```

```

def __init__(self, content: str, prompt_tokens: int = 0,
completion_tokens: int = 0):

```

```
self.content = content
self.prompt_tokens = prompt_tokens
self.completion_tokens = completion_tokens
```

```
class _GeminiLLM:
```

```
def __init__(
    self, model: str, temperature: float, max_tokens: int,
    thinking_budget: int = 0, json_mode: bool = False, ):
    self.model_name = model
    self.temperature = temperature
    self._max_tokens = max_tokens
    self._thinking_budget = thinking_budget
    self._json_mode = json_mode
```

```
def invoke(self, messages: list[BaseMessage]) ->
    _Response:
    system_content: str | None = None
    contents: list[google_types.Content] = []
```

```
for msg in messages:
    if isinstance(msg, SystemMessage):
        system_content = msg.content
    elif isinstance(msg, HumanMessage):
        contents.append(
            google_types.Content(role="user",
                parts=[google_types.Part(text=msg.content)])
        )
    elif isinstance(msg, AIMessage):
        contents.append(
```



```
google_types.Content(role="model",
parts=[google_types.Part(text=msg.content)])
)
```

```
thinking_cfg = (
google_types.ThinkingConfig(thinking_budget=self._thinking_budget)
if self._thinking_budget > 0
else None
)
```

```
config = google_types.GenerateContentConfig(
system_instruction=system_content,
temperature=self.temperature,
max_output_tokens=self._max_tokens,
response_mime_type="application/json" if
self._json_mode else None,
thinking_config=thinking_cfg, )
```

```
max_attempts = 3
for attempt in range(max_attempts):
try:
response = _client.models.generate_content(
model=self.model_name, contents=contents,
config=config, )
usage = response.usage_metadata
return _Response(
content=response.text or "",
prompt_tokens=usage.prompt_token_count or 0,
completion_tokens=usage.candidates_token_count or 0, )
```

```

except Exception as e:
    err = str(e).lower()
    is_retryable = any(k in err for k in [ "429", "quota", "rate
limit", "resource exhausted", "503", "unavailable",
"overloaded", "high demand", "500", "internal", ])
    if is_retryable:
        if attempt == max_attempts - 1:
            raise _retry_sleep(attempt, e)
        else:
            raise

def get_model_name(agent_id: str) -> str:
    return _MODEL_MAP[agent_id]
def get_llm(agent_id: str) -> _GeminiLLM:
    return _GeminiLLM(
        model=_MODEL_MAP[agent_id],
        temperature=_TEMPERATURE_MAP[agent_id],
        max_tokens=_MAX_TOKENS_MAP[agent_id],
        thinking_budget=_THINKING_BUDGET_MAP[agent_id
    ], json_mode=_JSON_MODE_MAP[agent_id],
    )
_llm_cache: dict[str, _GeminiLLM] = {}
def get_cached_llm(agent_id: str) -> _GeminiLLM:
    if agent_id not in _llm_cache: _llm_cache[agent_id] =
        get_llm(agent_id)
    return _llm_cache[agent_id]

```